

Module 1 – Topic 1: Introduction to Software

Q1. What is Software? (Include IEEE Definition)

Answer:

IEEE Definition:

"Software is the collection of computer programs, procedures, rules and associated documentation and data."

Software acts as an interface between the user and computer hardware. It provides instructions that enable the hardware to perform various tasks.

Examples: Windows, MS Word, Google Chrome, WhatsApp.

Q2. Explain the Components of Software.

Answer:

Software consists of the following components:

1. Programs

A program is a set of instructions written in a programming language to perform specific tasks.

Example: MS Word, Google Chrome.

2. Procedures

Procedures are step-by-step instructions that explain how to install, operate, and use the software.

Example: Installation guide, User manual.

3. Rules

Rules define how the software should function and the conditions it must follow.

Example:

- Password must contain at least 8 characters.
 - OTP is valid for only 2 minutes.
-

4. Documentation

Documentation includes all written information related to the software, such as user manuals, technical documents, and maintenance guides.

Example: User Guide, API Documentation.

5. Data

Data refers to the information that is stored, processed, and managed by the software.

Example: Customer records, Employee details, Student database.

Q3. Differentiate between Generic Software and Bespoke Software with Examples.

Answer:

Generic Software

1. Developed for a large number of users.
2. Sold in the general market.
3. Same features are available for all users.
4. Ready-made software.
5. Lower development cost per user.
6. Less development time.
7. Updates are provided to all users.
8. **Examples:** MS Word, Google Chrome, VLC Media Player.

Bespoke (Custom) Software

1. Developed for a specific customer or organization.
2. Developed according to the customer's requirements.
3. Features are customized for the client.
4. Built from scratch for a particular customer.
5. Higher development cost due to customization.
6. More development time.
7. Updates are provided according to customer requirements.
8. **Examples:** Hospital Management System, Banking Software, College ERP System.

Q4. Explain the Characteristics of Software.

Answer:

Software has the following characteristics:

1. **Software is developed, not manufactured.**
 - Software is created through design, coding, testing, and maintenance.
 - Unlike hardware, it is not produced in factories.
2. **Software does not wear out.**
 - Software does not physically deteriorate with time.
 - It may become obsolete due to changing requirements, bugs, or technology updates.
3. **Software is intangible.**
 - Software cannot be seen or touched physically.
 - It exists in digital form.
4. **Software is easy to modify.**
 - It can be updated, improved, and maintained to meet new requirements.
5. **Software is highly complex.**
 - Modern software consists of millions of lines of code and requires systematic development.
6. **Software can be reused.**
 - Existing software components or modules can be reused in different applications.
7. **Software requires continuous maintenance.**
 - Errors are corrected, security is improved, and new features are added after deployment.
8. **Most software is custom-built or engineered.**
 - Many software systems are developed to meet the specific needs of users or organizations.

Q5. Why is software developed rather than manufactured?

Answer:

Software is **developed rather than manufactured** because it is created through engineering activities such as analysis, design, coding, testing, and maintenance.

Unlike hardware, software is not assembled from physical components in a factory. Once developed, it can be copied and distributed at a very low cost without manufacturing it again.

Key Points:

- It is created through software development processes.
- It does not require physical assembly.
- Copies can be made easily at a very low cost.
- Development cost is high, but reproduction cost is very low.

Q6. Why does software not wear out like hardware?

Answer:

Software does not wear out because it has **no physical components** that can deteriorate with use.

Hardware components such as keyboards, hard disks, and processors wear out over time due to physical usage. Software, however, remains unchanged unless it is modified.

Although software does not wear out physically, it may require maintenance because of:

- Bugs or errors
- Changing user requirements
- Technology upgrades
- Security vulnerabilities

Thus, software becomes **obsolete** rather than wearing out.

Q7. Explain the Attributes of Good Software.

Answer:

Good software should possess the following attributes:

1. Maintainability

Software should be easy to modify, update, and repair according to changing user requirements.

2. Dependability

Software should be reliable, secure, and safe. It should perform its intended functions correctly without frequent failures.

3. Efficiency

Software should make effective use of system resources such as CPU, memory, storage, and network while providing good performance.

4. Usability

Software should be easy to learn, understand, and use. It should provide a user-friendly interface.

These attributes ensure that software is reliable, efficient, and satisfies user needs.

Q8. Explain Maintainability.

Answer:

Maintainability is the ability of software to be modified, updated, corrected, or enhanced after it has been deployed.

It includes:

- Fixing bugs
- Adding new features
- Improving performance
- Adapting to changing requirements

Example: WhatsApp regularly releases updates to fix bugs and introduce new features.

Q9. Explain Dependability.

Answer:

Dependability refers to the ability of software to perform its intended functions reliably, securely, and safely.

A dependable software system should:

- Produce accurate results

- Protect user data
- Minimize failures
- Be available when needed

Example: Banking software should process transactions correctly and securely.

Q10. Explain Efficiency.

Answer:

Efficiency is the ability of software to provide good performance while making optimal use of system resources such as CPU, memory, storage, and network.

Efficient software:

- Executes tasks quickly
- Uses less memory
- Consumes fewer system resources

Example: Google Chrome is designed to load web pages quickly while efficiently managing system resources.

Q11. Explain Usability.

Answer:

Usability is the ease with which users can learn, operate, and interact with software.

Usable software should:

- Have a simple and attractive interface
- Be easy to learn
- Be easy to navigate
- Help users perform tasks efficiently

Example: WhatsApp has a simple interface, making it easy for users of all ages to send messages, make calls, and share media.

Q12. What is Software Engineering? (Include IEEE Definition)

Answer:

Software Engineering is the application of engineering principles, methods, and tools to develop, operate, maintain, and manage software efficiently.

It aims to produce high-quality software that is reliable, efficient, maintainable, and cost-effective.

IEEE Definition

"Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."

Example: Developing a banking application by following proper planning, analysis, design, coding, testing, and maintenance.

Q13. Explain the Need/Objectives of Software Engineering.

Answer:

The main objectives (or need) of Software Engineering are:

1. **To develop high-quality software** that meets user requirements.
 2. **To reduce development cost** by using systematic methods.
 3. **To complete projects on time** by following proper planning.
 4. **To improve software reliability** by reducing errors and failures.
 5. **To simplify maintenance** so that software can be easily updated and modified.
 6. **To manage large and complex projects** efficiently.
 7. **To improve productivity** using standard tools and techniques.
 8. **To ensure customer satisfaction** by delivering software that fulfills user needs.
-

Q14. Explain the Scope of Software Engineering.

Answer:

The scope of Software Engineering covers all activities involved in the development and maintenance of software.

It includes:

1. **Requirement Analysis** – Understanding user needs.
2. **System Design** – Designing the software architecture and components.
3. **Implementation (Coding)** – Writing the program code.
4. **Testing** – Detecting and removing defects.

5. **Deployment** – Installing the software for users.
6. **Maintenance** – Correcting errors and adding new features after deployment.
7. **Project Management** – Planning, scheduling, and controlling software projects.
8. **Quality Assurance** – Ensuring that the software meets quality standards.

Q15. Explain the Software Engineering Process (All Stages).

Answer:

The **Software Engineering Process** is a systematic sequence of activities followed to develop high-quality software. It consists of the following stages:

1. Planning

- Identify project objectives and scope.
- Estimate cost, time, and resources.
- Prepare the project schedule.

2. Requirement Analysis

- Gather and analyze user requirements.
- Prepare the Software Requirement Specification (SRS) document.

3. Design

- Design the software architecture, database, user interface, and modules.
- Create the blueprint of the software.

4. Implementation (Coding)

- Convert the design into source code using programming languages.

5. Testing

- Test the software to identify and remove errors.
- Ensure that the software works according to user requirements.

6. Deployment

- Install and release the software for users.

7. Maintenance

- Fix bugs, improve performance, and add new features after deployment.

Q16. Explain Verification and Validation.

Answer:

Verification

Verification is the process of checking whether the software is being developed **according to the specified requirements and design**.

- It is a **static process**.
- It is performed **without executing the program**.
- It ensures that the product is built correctly.

Examples:

- Reviews
 - Inspections
 - Walkthroughs
-

Validation

Validation is the process of checking whether the **developed software meets the user's needs**.

- It is a **dynamic process**.
- It is performed **by executing the software**.
- It ensures that the correct product has been built.

Examples:

- Unit Testing
 - Integration Testing
 - System Testing
 - Acceptance Testing
-

Q17. Differentiate between Verification and Validation.

Verification

Checks whether the software is built correctly.

Static process.

Does not execute the software.

Validation

Checks whether the correct software is built.

Dynamic process.

Executes the software.

Verification

Performed during development.

Detects defects early.

Includes reviews, inspections, and walkthroughs.

Focuses on documents and design.

Objective: Prevent defects.

Validation

Performed after development/testing.

Ensures user requirements are satisfied.

Includes unit, integration, system, and acceptance testing.

Focuses on the final software product.

Objective: Detect defects and ensure customer satisfaction.

Q18. Explain the Importance of Software Engineering.**Answer:**

Software Engineering plays an important role in developing reliable, efficient, and high-quality software. Its importance is as follows:

1. Handling Large Projects

Software Engineering provides a systematic approach to manage and develop large and complex software projects efficiently.

2. Reduces Development Cost

Proper planning, design, and resource management help reduce unnecessary expenses and overall development cost.

3. Saves Time

Following a structured development process ensures that software is completed within the scheduled time.

4. Produces Reliable Software

Testing and quality assurance help develop software that is reliable, secure, and free from major defects.

5. Improves Effectiveness

Software developed according to engineering standards performs efficiently and meets user requirements.

6. Reduces Complexity

Large software projects are divided into smaller modules, making them easier to develop, test, and maintain.

7. Increases Productivity

Using standard methods, tools, and testing techniques improves the productivity of the development team.

8. Easy Maintenance

Software can be updated, modified, and enhanced easily after deployment, ensuring a longer software life.

9. Improves Customer Satisfaction

Developing software according to user requirements results in higher customer satisfaction and better software quality.

10. Ensures Better Quality

Software Engineering follows quality standards and best practices, resulting in software that is efficient, maintainable, and dependable.

Q19. Explain Modularity.

Answer:

Modularity is the process of dividing a software system into smaller, independent modules. Each module performs a specific function and can be developed, tested, and maintained separately.

Advantages:

- Reduces complexity.
- Easier testing and debugging.
- Simplifies maintenance.
- Promotes code reuse.

Q20. Explain Abstraction.

Answer:

Abstraction is the process of hiding the internal implementation details and showing only the essential features of a software component.

Advantages:

- Reduces complexity.
- Improves security.
- Makes software easier to understand.
- Simplifies maintenance.

Example: While using an ATM, the user only sees options like "Withdraw" and "Balance Inquiry" without knowing the internal processing.

Q21. Explain Encapsulation.

Answer:

Encapsulation is the process of combining data and the methods that operate on it into a single unit (class) while restricting direct access to the data.

Advantages:

- Protects data from unauthorized access.
- Improves security.
- Makes code easier to maintain.
- Promotes data hiding.

Example: In a Bank Account class, the account balance is accessed only through methods like deposit() and withdraw().

Q22. Explain Reusability.

Answer:

Reusability is the ability to use existing software components, modules, or code in multiple projects without rewriting them.

Advantages:

- Saves development time.
- Reduces development cost.
- Increases productivity.
- Improves software quality.

Example: A login module used in different applications.

Q23. Explain Maintenance.

Answer:

Maintenance is the process of modifying software after deployment to correct defects, improve performance, or add new features.

Types of Maintenance:

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance

- Preventive Maintenance

Advantages:

- Improves software quality.
- Extends software life.
- Keeps software updated.

Note: Your Unit 1 PPT mentions maintenance as a principle but **does not discuss the four types**. The types above are standard Software Engineering knowledge that many universities expect.

Q24. Explain Testing.

Answer:

Testing is the process of checking software to identify errors and ensure that it meets the specified requirements.

Objectives:

- Detect defects.
- Improve software quality.
- Verify functionality.
- Ensure reliability.

Example: Testing a shopping application before it is released to customers.

Q25. Explain Design Patterns.

Answer:

Design Patterns are reusable solutions to commonly occurring software design problems. They provide standard approaches for designing flexible and maintainable software.

Advantages:

- Improves code quality.
- Promotes reusability.
- Simplifies maintenance.
- Saves development time.

Q26. Explain Agile Methodology (Basic Introduction).

Answer:

Agile Methodology is a software development approach that focuses on iterative development, customer collaboration, flexibility, and continuous improvement.

Features:

- Incremental development.
- Customer involvement.
- Quick delivery.
- Easy adaptation to changing requirements.

Note: Unit 1 only mentions Agile as a principle. The detailed explanation of Agile belongs to **Unit 2**, where you'll study Scrum, XP, and Agile Process Models.

Q27. Explain Continuous Integration and Continuous Deployment (CI/CD).

Answer:

Continuous Integration (CI)

Continuous Integration is the practice of regularly merging code changes into a shared repository, where automated testing is performed to detect errors early.

Continuous Deployment (CD)

Continuous Deployment is the practice of automatically deploying tested software changes to the production environment, making new features available to users quickly.

Advantages of CI/CD:

- Faster software delivery.
- Early detection of errors.
- Improved software quality.
- Reduced deployment time.

Q28. Explain the Categories of Software.

Answer:

Software can be classified into the following categories:

1. System Software

System software manages computer hardware and provides a platform for application software to run.

Examples:

- Windows
 - Linux
 - Android
 - Device Drivers
-

2. Application Software

Application software is designed to help users perform specific tasks.

Examples:

- MS Word
 - Google Chrome
 - WhatsApp
 - VLC Media Player
-

3. Native Software

Native software is developed for a specific operating system or platform.

Examples:

- Safari (macOS/iOS)
 - Apple GarageBand (macOS)
-

4. Cross-platform Software

Cross-platform software is designed to run on multiple operating systems without major modifications.

Examples:

- Google Chrome
- Firefox

- VLC Media Player
- Visual Studio Code

Q29. Differentiate between System Software and Application Software.

Answer:

System Software	Application Software
1. Manages computer hardware and system resources.	Helps users perform specific tasks.
2. Acts as a platform for application software.	Runs on top of system software.
3. Starts automatically when the computer boots.	Runs only when the user opens it.
4. Essential for system operation.	Not essential for booting the system.
5. Works in the background.	Directly interacts with users.
6. Usually developed by OS manufacturers.	Developed by software companies or developers.
7. Examples: Windows, Linux, Android.	Examples: MS Word, Chrome, Photoshop.
8. Controls hardware operations.	Performs user-specific operations.

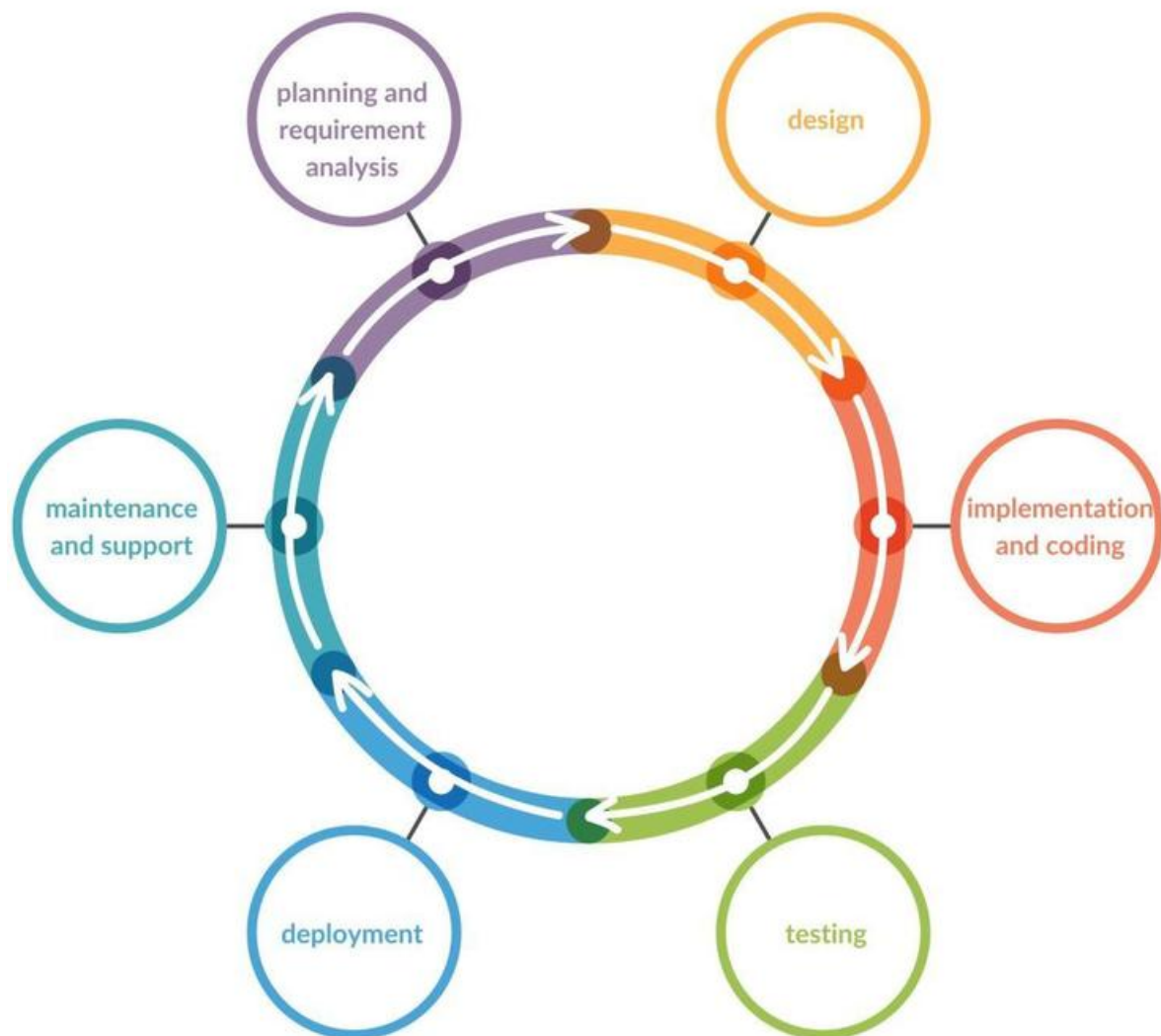
Q30. Differentiate between Native Software and Cross-platform Software.

Answer:

Native Software	Cross-platform Software
1. Developed for a specific operating system.	Developed to run on multiple operating systems.
2. Optimized for one platform.	Works across different platforms.
3. Better performance on its target platform.	Performance may vary across platforms.
4. Requires separate versions for different operating systems.	A single codebase can support multiple operating systems.
5. Uses platform-specific features.	Uses common frameworks and libraries.
6. Development cost is higher for multiple platforms.	Development cost is lower due to code reuse.
7. Examples: Safari, GarageBand.	Examples: Chrome, VLC, VS Code.
8. Platform dependent.	Platform independent.

ALSO from PPT

31. Draw and Explain the Software Engineering Process.



Stages of the Software Engineering Process

1. Planning

- In this stage, the project objectives, scope, budget, timeline, and required resources are identified.
- A project plan is prepared before development begins.

2. Requirement Analysis

- The requirements of the customer are collected, analyzed, and documented.
- These requirements are recorded in the **Software Requirement Specification (SRS)** document.

3. Design

- Based on the requirements, the software architecture, database, user interface, and system modules are designed.
- This serves as the blueprint for software development.

4. Implementation (Coding)

- The software design is converted into source code using programming languages.
- Each module is developed according to the design specifications.

5. Testing

- The developed software is tested to identify and remove defects.
- It ensures that the software functions correctly and satisfies user requirements.

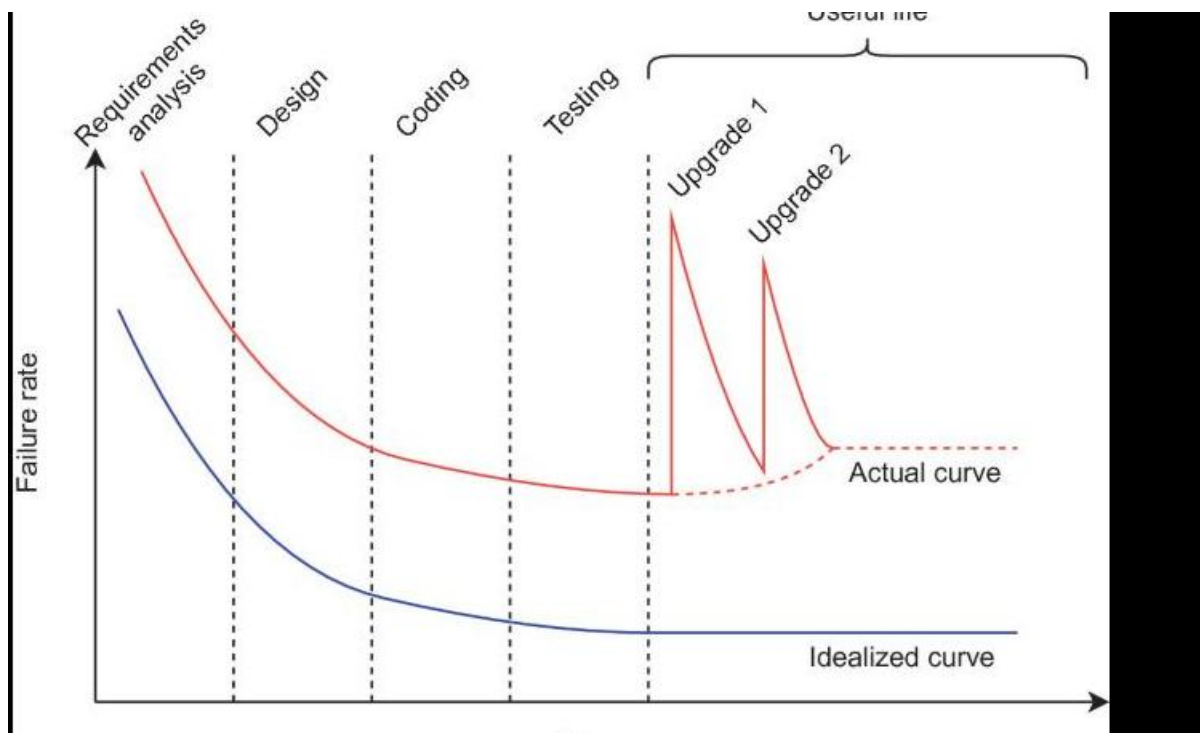
6. Deployment

- After successful testing, the software is installed and released for use by the customers.
- Users can now access and use the software.

7. Maintenance

- After deployment, the software is continuously monitored.
- Bugs are fixed, performance is improved, and new features are added according to user needs.

Q32. Draw and Explain the Software Failure Curve



The **Software Engineering Process** is a systematic sequence of activities followed to develop, test, deploy, and maintain software. It helps in producing high-quality software that meets user requirements efficiently.

Q32. Draw and Explain the Software Failure Curve

1. Initial Phase

- At the beginning of software usage, the **failure rate is high** because many defects or bugs are still present.
- As users and testers identify these defects, developers fix them through testing and debugging.
- Therefore, the failure rate gradually starts decreasing.

2. Stable Phase

- After most major defects have been corrected, the software becomes **stable and reliable**.
- The failure rate remains **low and almost constant** because the software performs as expected.
- Only a few minor errors may occur during this stage.

3. Modified Phase

- When new features, updates, or modifications are added to the software, **new bugs may be introduced**.
 - Due to these changes, the failure rate may **increase temporarily**.
 - Additional testing is required to identify and fix the newly introduced defects.
-

4. Maintenance Phase

- After fixing the new defects, the software becomes stable again.
- Regular maintenance, updates, and improvements help **reduce the failure rate** and improve software quality.
- This process continues throughout the software's life cycle to keep it secure, efficient, and reliable.